

OutPut:

default

POST /AddMusic Post Request

Parameters

No parameters

Request body *expand*

application/json

Full Value | Schema

```
{
  "Name": "Naa Ready Tha",
  "Director": "Anirudh",
  "Singer": "Thalapathy"
}
```

Cancel Clear

Response

Curl

```
-x 'POST' \
  'http://127.0.0.1:8001/AddMusic' \
  -d '{"Name": "Naa Ready Tha",
    "Director": "Anirudh",
    "Singer": "Thalapathy"}' \
  -H 'accept: application/json'
```

Request URL

http://127.0.0.1:8001/AddMusic

Server response

Code Details

200

Response body

```
{
  "message": "Music added successfully"
}
```

Download

Response headers

```
content-length: 66
content-type: application/json
date: Mon, 18 Aug 2025 12:05:51 GMT
server: uvicorn
```

Response

GET /GetAllMusic Getmusic

Parameters

No parameters

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8001/GetAllMusic' \
  -H 'accept: application/json'
```

Request URL

http://127.0.0.1:8001/GetAllMusic

Server response

Code Details

200

Response body

```
{
  {
    "Name": "Naa Ready Tha",
    "Director": "Anirudh",
    "Singer": "Thalapathy"
  }
}
```

Download

Response headers

```
content-length: 69
content-type: application/json
date: Mon, 18 Aug 2025 11:59:32 GMT
server: uvicorn
```

GET /GetMusicByName Getmusicbyname

Parameters

Name	Description
Name * required string (query)	Naa

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  http://127.0.0.1:8001/GetMusicByName?Name=NaaK20' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8001/GetMusicByName?Name=NaaK20
```

Server response

Code	Details
200	Response body <pre>{ "Name": "Naa Ready Tha", "Director": "Anirudh", "Singer": "Thalapathy" }</pre> Response headers <pre>content-length: 67 content-type: application/json date: Mon, 18 Aug 2025 12:00:49 GMT server: uvicorn</pre>

App.py:

```
from fastapi import FastAPI, status, Depends, Response
import uvicorn
import MusicSql as sql
from sqlalchemy.orm import Session

app = FastAPI()

@app.post("/AddMusic")
async def Post_student(music: sql.Music, db: Session =
Depends(sql.get_db)):
    sql.add_music(music.Name, music.Director, music.Singer, db)
    return {"message": "Music added successfully"}

@app.get("/GetAllMusic")
async def GetMusic(db: Session = Depends(sql.get_db)):
    return sql.GetAllMusic(db)

@app.get("/GetMusicByName")
async def GetMusicByName(Name: str, response: Response, db: Session =
Depends(sql.get_db)):
```

```
music = sql.GetMusicByName(db, Name)
if music:
    return music
response.status_code = status.HTTP_404_NOT_FOUND
return { "Message": "Not Found" }

if __name__ == "__main__":
    uvicorn.run("MusicFast:app", host="127.0.0.1", port=8001)
```

[sql.py](#):

```
from fastapi import Depends
from pydantic import BaseModel
from sqlalchemy import create_engine, func
from sqlalchemy.dialects.sqlite import *
from sqlalchemy.orm import sessionmaker, declarative_base, Session
url = "sqlite:///./Training.db"
engine = create_engine(url, connect_args={"check_same_thread": False})
session = sessionmaker(autocommit=False, autoflush=False, bind=engine)

Base = declarative_base()

from sqlalchemy import Column, Integer, String
class Musics(Base):
    __tablename__ = "Music"
    ID = Column(Integer, primary_key=True, index = True)
    Name = Column(String(50), nullable = False)
    Singer = Column(String(50), nullable = False)
    Director = Column(String(50), nullable = False)

Base.metadata.create_all(bind=engine)

class Music(BaseModel):
    Name: str
    Director: str
    Singer: str
    class Config:
        orm_mode = True
        from_attributes=True

def get_db():
    db = session()
    try:
        yield db
    finally:
        db.close()
```

```
def add_music(name,director,singer,db: Session):
    st = Musics(Name = name ,Director = director,Singer = singer)
    db.add(st)
    db.commit()
    db.refresh(st)

def GetAllMusic(db:Session):
    MusicList = db.query(Musics).all()
    Music_schemas = [Music.model_validate(stud) for stud in MusicList]
    return Music_schemas

def GetMusicByName(db:Session,name:str):
    music =
db.query(Musics).filter(func.lower(Musics.Name).contains(name.lower())).fi
rst()
    if music:
        return Music.model_validate(music)
    return None
```